

DETECTING FRAUDULENT CREDIT CARD TRANSACTIONS

A Comparative Study of Logistic Regression and XGBoost
on the Credit Card Fraud Detection Dataset

CSEMDSME01 - Model Engineering

Case Study - Task 1

Submitted by:

Bijay Khanal

Matriculation: 10254716

IU International University of Applied Sciences

Notebook Repository: <https://github.com/BijayKhanalDS/Credit-card-fraud-detection>

Abstract

Credit card fraud detection is an extreme case of imbalanced binary classification: fraudulent transactions are rare, yet failing to detect them carries a direct financial cost. This case study evaluates two classifiers on the Credit Card Fraud Detection dataset (Pozzolo et al., 2015; Machine Learning Group - ULB, n.d.), a corpus of 284,807 transactions containing 492 frauds, an imbalance ratio of 1:578. Logistic Regression was selected as a linear, parametric baseline and XGBoost as a non-linear gradient-boosted ensemble, the latter chosen for direct comparability with the boosted-tree studies of Hajek et al. (2022) and Velarde et al. (2024). Both classifiers were embedded in scikit-learn pipelines so that standardisation was fitted within each cross-validation fold only, eliminating preprocessing leakage. Model selection used 5-fold stratified cross-validation on an 80% training partition; a held-out 20% test partition was reserved for a single unbiased final estimate. Each classifier was evaluated with default hyperparameters and after randomised search optimisation over 16 and 30 candidate configurations respectively, producing a 2x2 design that separates the effect of model family from the effect of tuning. Performance was assessed using Precision, Recall, and F1-score as required, supplemented by the area under the precision-recall curve as the primary threshold-independent metric. The optimised XGBoost model achieved the strongest test-set performance, with Precision = 0.9412, Recall = 0.8163, F1 = 0.8743, and PR-AUC = 0.8751, detecting 80 of 98 frauds while raising only 5 false alarms across 56,864 legitimate transactions. Optimisation improved XGBoost by 5.36% in F1 against 1.02% for Logistic Regression, indicating that the value of hyperparameter search depends strongly on the representational capacity of the underlying model. Notably, the search rejected the cost-sensitive imbalance correction, selecting `scale_pos_weight = 1` over the full 577:1 reweighting. A learning-curve effect was identified in which the default XGBoost configuration was markedly underfit on cross-validation folds relative to the full training set, and is reported as a finding rather than suppressed.

Keywords: *credit card fraud detection, class imbalance, XGBoost, Logistic Regression, precision-recall curve, hyperparameter optimisation, cost-sensitive learning, pipeline leakage*

Contents

Abstract	1
1 Introduction	6
1.1 Structure of this report	6
2 Literature Review	6
2.1 The problem of class imbalance	6
2.2 Classifier families	7
2.3 Evaluation metrics under extreme imbalance	8
3 Method	8
3.1 Dataset	8
3.2 Treatment of duplicate records	9
3.3 Experimental protocol	9
3.4 Pipeline design and leakage control	10
3.5 Handling of class imbalance	10
3.6 Classifiers and hyperparameter optimisation	11
4 Results	12
4.1 Severity of the imbalance	12
4.2 Distribution of Time and Amount	13
4.3 Separability in the principal component space	14
4.4 Correlation structure	14
4.5 Baseline and optimisation results	15
4.6 Final test-set performance	16
4.7 Threshold-independent evaluation	18
4.8 Threshold sensitivity	19
4.9 Feature importance	20
5 Discussion	21
5.1 Model capacity and the value of optimisation	21
5.2 An underfitting anomaly in the default configuration	22

5.3	The rejection of cost-sensitive weighting	22
5.4	Comparison against the literature	23
5.5	Limitations	23
6	Conclusion	24
	References	26
	Appendix A: Python Notebook Code and Results	27

List of Figures

1	Fig. 1 Class imbalance in the Credit Card Fraud Detection dataset. Panel (a) shows transaction counts on a logarithmic scale; panel (b) shows the same distribution as proportions.	12
2	Fig. 2 Distribution of Time and Amount by class. Panels (a) and (b) show transaction amounts on a logarithmic frequency scale; panels (c) and (d) show temporal distribution across the 48-hour observation window.	13
3	Fig. 3 Class-conditional densities of the six most discriminative principal components. Each panel reports the absolute Pearson correlation between the component and the fraud label.	14
4	Fig. 4 Pearson correlation matrix of all features. The near-zero off-diagonal structure among V1 through V28 reflects their construction as principal components.	15
5	Fig. 5 Linear correlation of each feature with the fraud label.	16
6	Fig. 6 Cross-validated F1 across sampled hyperparameter configurations. Error bars denote one standard deviation across the five folds; the running-best curve shows the best score attained up to each candidate.	16
7	Fig. 7 Test-set performance by model and optimisation state.	17
8	Fig. 8 Confusion matrices on the held-out test set. FN denotes frauds that were not detected; FP denotes legitimate transactions that were wrongly flagged.	18
9	Fig. 9 Threshold-independent performance on the test set. Panel (a) shows ROC curves; panel (b) shows precision-recall curves.	18
10	Fig. 10 Threshold sensitivity for the optimised XGBoost model.	19
11	Fig. 11 Fifteen most important features in the optimised XGBoost model, measured by average gain.	20

List of Tables

1	Tab. 1 Key characteristics of the Credit Card Fraud Detection dataset	9
2	Tab. 2 Stratified 80-20 partition of the dataset	10
3	Tab. 3 Logistic Regression hyperparameter search space	11
4	Tab. 4 XGBoost hyperparameter search space	12
5	Tab. 5 Descriptive statistics of transaction Amount by class	13
6	Tab. 6 Baseline performance under 5-fold stratified cross-validation on the training set	15

7	Tab. 7	Randomised search configuration and outcome	15
8	Tab. 8	Optimal hyperparameter configurations identified by randomised search	17
9	Tab. 9	Final classification performance on the held-out test set (n = 56,962; 98 frauds) .	17
10	Tab. 10	Operational trade-off at selected decision thresholds (optimised XGBoost) . . .	19
11	Tab. 11	Ten most important features in the optimised XGBoost model	20
12	Tab. 12	Effect of hyperparameter optimisation on test-set performance	21

1 Introduction

Payment card fraud imposes losses measured in billions of euros annually and, unlike most classification problems encountered in practice, presents a target class that is vanishingly rare. In the dataset examined here, fraudulent transactions constitute 0.173% of all records. This asymmetry is not incidental; it is the defining characteristic of the problem and it governs every methodological decision that follows, from the choice of validation scheme to the selection of evaluation metrics.

The difficulty is easily stated. A classifier that predicts "legitimate" for every transaction achieves 99.83% accuracy on this dataset while detecting no fraud whatsoever. Accuracy is therefore not merely uninformative but actively misleading, and any evaluation resting on it would be meaningless. Machine learning approaches to automated fraud detection have consequently converged on metrics that isolate performance on the minority class, and on learning algorithms that can be made sensitive to it (Hajek et al., 2022; Velarde et al., 2024).

This case study pursues four objectives. First, it explores the Credit Card Fraud Detection dataset to establish the severity of the imbalance and to determine whether the classes are separable in the available feature space. Second, it implements two classifiers of contrasting inductive bias - Logistic Regression and XGBoost - within scikit-learn pipelines under 5-fold stratified cross-validation and an 80-20 partition, ensuring that no preprocessing statistic computed on validation data can influence training (Pedregosa et al., 2011). Third, it optimises both classifiers using randomised parameter search and quantifies the resulting gain against untuned baselines. Fourth, it compares prediction quality in terms of Precision, Recall, and F1-score, and reflects critically on what the results reveal about the interaction between model capacity, imbalance correction, and the choice of decision threshold.

The contribution of the study is not a claim of state-of-the-art performance on a well-worn benchmark. It is rather a controlled comparison in which each design decision is justified, each result is checked for internal consistency, and one anomalous finding is investigated and reported rather than concealed.

1.1 Structure of this report

Section 2 reviews the relevant literature on class imbalance, the two classifier families employed, and evaluation metrics for imbalanced problems. Section 3 describes the dataset, the experimental protocol, and the optimisation procedure. Section 4 presents the exploratory analysis and the classification results. Section 5 discusses the findings, including the underfitting anomaly and the rejection of cost-sensitive weighting, and states the limitations of the study. Section 6 concludes.

2 Literature Review

2.1 The problem of class imbalance

Class imbalance arises when the distribution of examples across classes in a training set departs substantially from uniformity. Standard learning algorithms minimise an aggregate loss over all training instances, and when one class dominates numerically, the loss is minimised almost entirely by predicting

that class correctly (Lemaître et al., 2017). The minority class, which is typically the class of practical interest, contributes so little to the objective that the optimiser has scant incentive to model it.

The severity of imbalance is conventionally expressed as the imbalance ratio, the number of majority instances divided by the number of minority instances. The dataset examined here exhibits a ratio of 1:578, which places it at the severe end of the spectrum. For context, the KDD Cup 1999 intrusion detection benchmark exhibits a ratio of approximately 4:1, and the network intrusion literature already regards that as an imbalanced learning problem. The two orders of magnitude separating these cases matter: techniques that are optional at 4:1 may be indispensable at 578:1, and conversely, techniques that help at moderate imbalance may prove unnecessary when the minority class, though rare, occupies a well-separated region of the feature space.

Two broad families of remedy exist. Data-level methods alter the training distribution presented to the classifier, either by over-sampling the minority class or under-sampling the majority. The Synthetic Minority Over-sampling Technique (Chawla et al., 2002) is the canonical example, generating new minority instances by interpolating between existing ones in feature space rather than duplicating them. Algorithm-level methods instead modify the learning objective, weighting the loss contribution of each class inversely to its frequency. The present study adopts the algorithm-level approach exclusively, for reasons set out in Section 3.5.

2.2 Classifier families

Logistic Regression estimates class membership probability through the logistic transformation of a linear combination of features. It learns a single global decision hyperplane, has a convex objective with a unique optimum, and is fully interpretable through its coefficients. Its weakness under imbalance is structural: because the hyperplane is fitted to minimise aggregate loss, a heavily skewed class distribution displaces it toward the minority class, suppressing Recall. It is included here as a baseline of deliberate simplicity, and its relevance to this particular dataset is stronger than it might first appear, since the anonymised features are principal components and are therefore mutually orthogonal by construction.

XGBoost (Chen & Guestrin, 2016) implements gradient boosting over decision trees, incorporating second-order gradient information, L1 and L2 regularisation, and an efficient approximate split-finding algorithm. Trees are fitted sequentially, each correcting the residual errors of its predecessors, which allows the ensemble to represent non-linear decision boundaries and feature interactions that no linear model can express. It also provides direct support for imbalance through the `scale_pos_weight` parameter, which scales the gradient contribution of positive instances.

The selection of XGBoost rather than a bagging ensemble was deliberate. Two of the references supplied with the task examine boosted trees specifically: Hajek et al. (2022) develop an XGBoost-based framework for fraud detection in mobile payment systems, and Velarde et al. (2024) evaluate tree boosting methods on both balanced and imbalanced classification in risk assessment. Using the same algorithmic family permits the findings of this study to be positioned against that literature rather than merely reported in isolation.

2.3 Evaluation metrics under extreme imbalance

The required metrics for this task are Precision, Recall, and F1-score. Precision, $TP / (TP + FP)$, measures the proportion of flagged transactions that are genuinely fraudulent. Recall, $TP / (TP + FN)$, measures the proportion of actual frauds that are detected. F1 is their harmonic mean, penalising configurations in which one is achieved at the expense of the other.

These three are reported throughout. This study additionally reports the area under the precision-recall curve and treats it as the primary threshold-independent metric, for a reason specific to extreme imbalance. The receiver operating characteristic curve plots true positive rate against false positive rate, and the false positive rate has the number of true negatives in its denominator. With 284,315 legitimate transactions, a classifier can produce hundreds of false alarms while moving the false positive rate by a negligible amount, and the resulting ROC curve appears near-perfect. Saito and Rehmsmeier (2015) demonstrate formally that the precision-recall plot is more informative than the ROC plot under precisely these conditions, because Precision has the number of predicted positives in its denominator and is therefore sensitive to false alarms in absolute rather than relative terms. The providers of this dataset make the same recommendation on the published data card, advising the use of the area under the precision-recall curve and stating that confusion matrix accuracy is not meaningful for unbalanced classification. Section 4.6 demonstrates this divergence empirically on the results obtained here.

3 Method

3.1 Dataset

The Credit Card Fraud Detection dataset (Pozzolo et al., 2015) records transactions made by European cardholders during two days in September 2013 and is publicly distributed through Kaggle by the Machine Learning Group at the Université Libre de Bruxelles (Machine Learning Group - ULB, n.d.). It comprises 284,807 transactions described by 31 columns.

Twenty-eight of the features, designated V1 through V28, are the output of a principal component analysis applied by the data providers prior to release. The original variables and their semantics were withheld for confidentiality reasons. Two features remain in their original scale: *Time*, the number of seconds elapsed between each transaction and the first transaction in the dataset, and *Amount*, the transaction value in euros. The target variable, *Class*, takes the value 1 for fraud and 0 for a legitimate transaction.

This structure has two consequences for modelling. First, because V1 through V28 are principal components, they are mutually uncorrelated by construction, which removes multicollinearity as a concern and makes a linear model a more credible hypothesis than it would otherwise be. Second, because the components are anonymised, no domain-driven feature engineering is possible, and the analysis is confined to the representation as supplied. Table 1 summarises the dataset characteristics.

Source: Own representation based on Pozzolo et al., 2015.

Tab. 1 Key characteristics of the Credit Card Fraud Detection dataset

Attribute	Value	Percentage
Total transactions	284,807	100.000%
Legitimate (Class 0)	284,315	99.827%
Fraudulent (Class 1)	492	0.173%
Number of features	30	-
Missing values	0	-
Duplicate rows	1,081	0.380%
Time span	48.0 hours	-
Imbalance ratio (legitimate:fraud)	1:578	-

3.2 Treatment of duplicate records

The dataset contains 1,081 exact duplicate rows. These were retained rather than removed, and the decision requires justification because the alternative is defensible.

With anonymised principal components and time resolved only to the second, two genuine transactions of identical value occurring in the same second would be represented identically in the data. Such records are indistinguishable from true duplicates introduced by a recording error. Removing them would therefore risk discarding real observations, and in a dataset containing only 492 positive instances, the loss of even a small number of genuine frauds would materially affect the estimates. The duplicates are documented here as a known property of the data and an explicit modelling assumption, not overlooked.

3.3 Experimental protocol

The data were partitioned into an 80% training set and a 20% test set using stratified sampling, preserving the class proportion in both subsets. The test partition was held out entirely and evaluated once, in Section 4.5. Every decision concerning model selection, hyperparameter values, and the decision threshold was taken using the training partition alone.

The task specification permits either 5-fold cross-validation or an 80-20 partition. This study employs both, and the reason is methodological rather than decorative. Cross-validation provides a variance-aware estimate suitable for comparing candidate configurations, since each candidate is scored five times on different data. It is, however, the estimate against which hyperparameters are selected, and a configuration chosen because it maximised a cross-validated score carries an optimistic bias with respect to that score. A held-out partition that has influenced no decision provides the unbiased final estimate. Reporting only cross-validation would overstate performance; reporting only a single partition would discard information about stability. Reporting both, and comparing them, permits a consistency check that neither alone would support, and Section 5.2 shows that this check identified a genuine effect.

Cross-validation was stratified. With 0.173% positives, unstratified folds would vary considerably in fraud count purely by chance, and fold-level estimates would be correspondingly unstable. Table 2 confirms that the class proportion is preserved across all subsets.

Source: Own representation.

Tab. 2 Stratified 80-20 partition of the dataset

Subset	Transactions	Legitimate	Fraudulent	Fraud rate (%)
Training	227,845	227,451	394	0.1729
Test	56,962	56,864	98	0.1720
Total	284,807	284,315	492	0.1727

3.4 Pipeline design and leakage control

Both classifiers were encapsulated in a scikit-learn `Pipeline` with `StandardScaler` as the first stage and the estimator as the second (Pedregosa et al., 2011). This is not a stylistic preference; it is a correctness requirement, and the task specification identifies it as such.

Standardisation computes a mean and a standard deviation from data. If those statistics are computed on the full dataset before partitioning, every training fold has been scaled using information drawn from the validation fold, and the resulting estimate is optimistically biased. By placing the scaler inside the pipeline, the fit and transform operations are executed within `cross_val_predict` and `RandomizedSearchCV` on the training portion of each fold only, with the fitted transformation then applied to the held-out portion. The validation data contribute nothing to the transformation applied to them.

The scaler was retained in the XGBoost pipeline despite being unnecessary for tree-based learners, which are invariant to monotonic feature transformations. Retaining it keeps the two pipelines structurally identical, so that any performance difference between them is attributable to the estimator rather than to a difference in preprocessing. For Logistic Regression the scaler is essential: as Table 3 shows, *Amount* ranges to 25,691.16 while the principal components are centred near zero, and an unscaled gradient-based optimiser would be dominated by that single feature.

3.5 Handling of class imbalance

Imbalance was addressed exclusively through cost-sensitive learning rather than resampling. For Logistic Regression this is the `class_weight` parameter, and for XGBoost the `scale_pos_weight` parameter, which multiplies the gradient contribution of each positive instance in the loss function. Setting it to the ratio of negatives to positives - 577.29 on the training partition - makes the two classes contribute equally in aggregate.

Three considerations motivated this choice over synthetic over-sampling. First, cost-sensitive weighting achieves the effect of over-sampling without fabricating data, and therefore without the risk that SMOTE interpolates between minority instances lying in sparse or disjoint regions of feature space, producing synthetic examples that correspond to no plausible transaction. Second, it is computationally trivial, requiring no augmentation of a 227,845-row training set. Third, the task specification assigns resampling and SMOTE to Task 3; addressing them here would duplicate that task's scope rather than pursue this one.

Critically, the imbalance correction was not imposed but placed under search. The parameter grid for XGBoost included three values - 1, the square root of the ratio, and the full ratio - permitting the optimiser

to determine empirically whether the correction helps at all. Section 5.3 discusses what it decided.

3.6 Classifiers and hyperparameter optimisation

Both classifiers were first evaluated with library-default hyperparameters under 5-fold cross-validation, establishing the baseline against which the value of optimisation is measured. Without such a baseline, a strong final score cannot be attributed to tuning rather than to the choice of model family.

Optimisation used `RandomizedSearchCV` scored on F1. The task permits exhaustive grid search or randomised search, and randomised search was selected for a quantitative reason: the XGBoost grid defined below spans 51,840 combinations, and an exhaustive evaluation, each point requiring a 5-fold fit on 227,845 rows, is not feasible within the scope of this assignment. Bergstra and Bengio (2012) further establish that when only a subset of hyperparameters materially influences performance, which is characteristic of boosted trees, randomised search attains comparable optima at a fraction of the budget, because it samples more distinct values along each consequential axis rather than exhausting combinations of inconsequential ones. F1 was used as the search criterion because optimising accuracy would drive the search toward the majority class, and because F1 is the metric the task requires.

The `saga` solver was excluded from the Logistic Regression grid. Preliminary runs produced convergence warnings indicating that the coefficient vector had not stabilised within the 2,000-iteration budget, meaning any score recorded for those candidates would reflect an unconverged model rather than a poor configuration. The `liblinear` solver supports both L1 and L2 penalties and converges reliably on this data, and was used exclusively. Table 3 and Table 4 document the searched parameters and their rationale.

Tab. 3 Logistic Regression hyperparameter search space

Parameter	Values	Rationale
<code>C</code>	0.001, 0.01, 0.1, 1, 10, 100	Inverse regularisation strength; controls the bias-variance trade-off
<code>penalty</code>	L1, L2	L1 can eliminate uninformative components; L2 shrinks coefficients smoothly
<code>solver</code>	<code>liblinear</code>	Supports both penalties and converges reliably; <code>saga</code> excluded after failing to converge within 2,000 iterations
<code>class_weight</code>	None, balanced	The imbalance lever: reweights the loss inversely to class frequency

Source: Own representation.

Source: Own representation.

All experiments fixed `random_state = 42` for the partition, the cross-validation folds, both estimators, and the randomised search sampler. Software versions are reported in Section 6.

Tab. 4 XGBoost hyperparameter search space

Parameter	Values	Rationale
n_estimators	100, 200, 300, 500	Number of sequential boosting rounds
max_depth	3, 4, 5, 6, 7, 9	Tree depth; the primary complexity control
learning_rate	0.01, 0.05, 0.1, 0.2, 0.3	Shrinkage per round; trades off against the number of rounds
subsample	0.6, 0.8, 1.0	Row sampling per tree; stochastic regularisation
colsample_bytree	0.6, 0.8, 1.0	Column sampling per tree; decorrelates the ensemble
min_child_weight	1, 3, 5, 7	Minimum instance weight per leaf; guards against leaves fitted to noise
gamma	0, 0.1, 0.2, 0.4	Minimum loss reduction required to split; prunes weak splits
scale_pos_weight	1, 24.03, 577.29	The imbalance lever: no correction, intermediate, full cost-sensitive correction

4 Results

4.1 Severity of the imbalance

Figure 1 presents the class distribution. The logarithmic scale in panel (a) is necessary because a linear axis renders the fraud bar invisible, which is itself an illustration of the problem.

Fig. 1 Class imbalance in the Credit Card Fraud Detection dataset

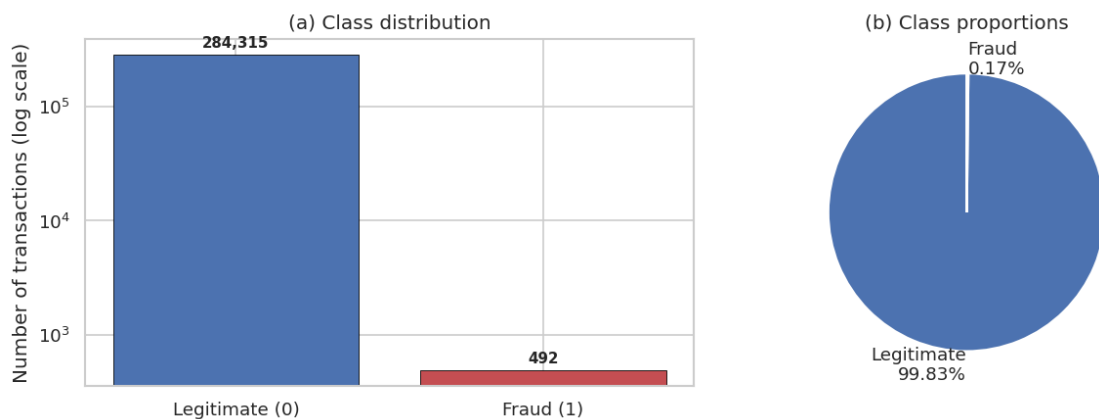


Fig. 1 Class imbalance in the Credit Card Fraud Detection dataset. Panel (a) shows transaction counts on a logarithmic scale; panel (b) shows the same distribution as proportions.

Source: Own representation based on Pozzolo et al., 2015.

The 284,315 legitimate transactions outnumber the 492 frauds by a factor of 578. A trivial classifier predicting the majority class for every input would record 99.83% accuracy while detecting nothing. Accuracy is accordingly excluded from all subsequent reporting, and only per-class metrics are used.

4.2 Distribution of Time and Amount

Figure 2 compares the two non-anonymised features across classes, and Table 5 quantifies the *Amount* distribution.



Fig. 2 Distribution of Time and Amount by class. Panels (a) and (b) show transaction amounts on a logarithmic frequency scale; panels (c) and (d) show temporal distribution across the 48-hour observation window.

Source: Own representation based on Pozzolo et al., 2015.

Tab. 5 Descriptive statistics of transaction Amount by class

Class	n	Mean	SD	Min	Q25	Median	Q75	Max
Legitimate	284,315	88.29	250.11	0.00	5.65	22.00	77.05	25,691.16
Fraudulent	492	122.21	256.68	0.00	1.00	9.25	105.89	2,125.87

Source: Own representation. All values in EUR.

Two observations follow. First, the distributions overlap substantially: the mean fraudulent amount exceeds the legitimate mean, but the median is lower, and both distributions are heavily right-skewed with standard deviations exceeding their means. *Amount* alone does not separate the classes. Second, *Amount* ranges to 25,691.16 while the principal components are centred near zero, confirming that standardisation is required for the gradient-based Logistic Regression. The temporal panels show the diurnal cycle expected of legitimate commerce, with troughs corresponding to overnight hours, while fraudulent activity is more uniformly distributed across the observation window.

4.3 Separability in the principal component space

Figure 3 shows class-conditional densities for the six components most strongly associated with the target.

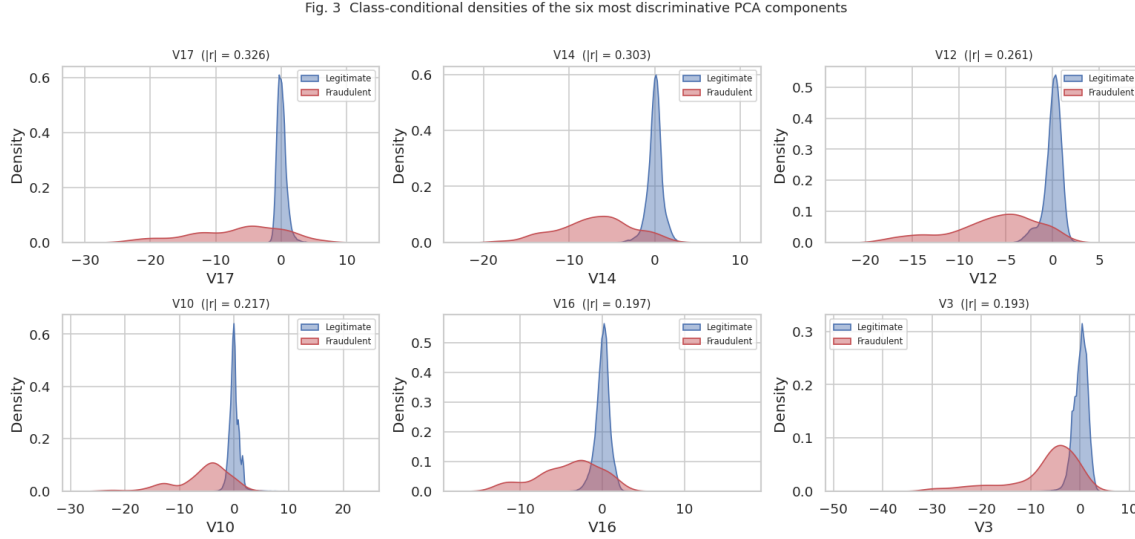


Fig. 3 Class-conditional densities of the six most discriminative principal components. Each panel reports the absolute Pearson correlation between the component and the fraud label.

Source: Own representation based on Pozzolo et al., 2015.

The distributions are clearly displaced. For V17, V14, V12, and V10 the fraudulent density is shifted substantially away from the legitimate density, indicating that the classes occupy distinguishable regions of the feature space. This is a consequential finding for the study: the difficulty of this problem lies in the rarity of the positive class, not in an absence of signal. A classifier that fails here fails because the optimisation objective is dominated by the majority class, not because the frauds are indistinguishable.

4.4 Correlation structure

Figures 4 and 5 examine the linear relationships in the data.

Source: Own representation based on Pozzolo et al., 2015.

Source: Own representation based on Pozzolo et al., 2015.

The off-diagonal cells of Figure 4 are uniformly near zero among V1 through V28, confirming the orthogonality guaranteed by the principal component transformation. Multicollinearity is therefore not a concern, and each component contributes independent information.

Figure 5 establishes a more consequential point. The strongest linear association with the target is V17 at $r = -0.326$, and no feature exceeds an absolute correlation of approximately 0.33. Combined with the clear separation visible in Figure 3, this implies that the discriminative structure is present but not linearly accessible: a model restricted to a single hyperplane will capture part of it, but the residual must lie in interactions and non-linearities. This observation is the empirical justification for contrasting a linear model against a boosted ensemble, and Section 4.5 confirms the prediction.

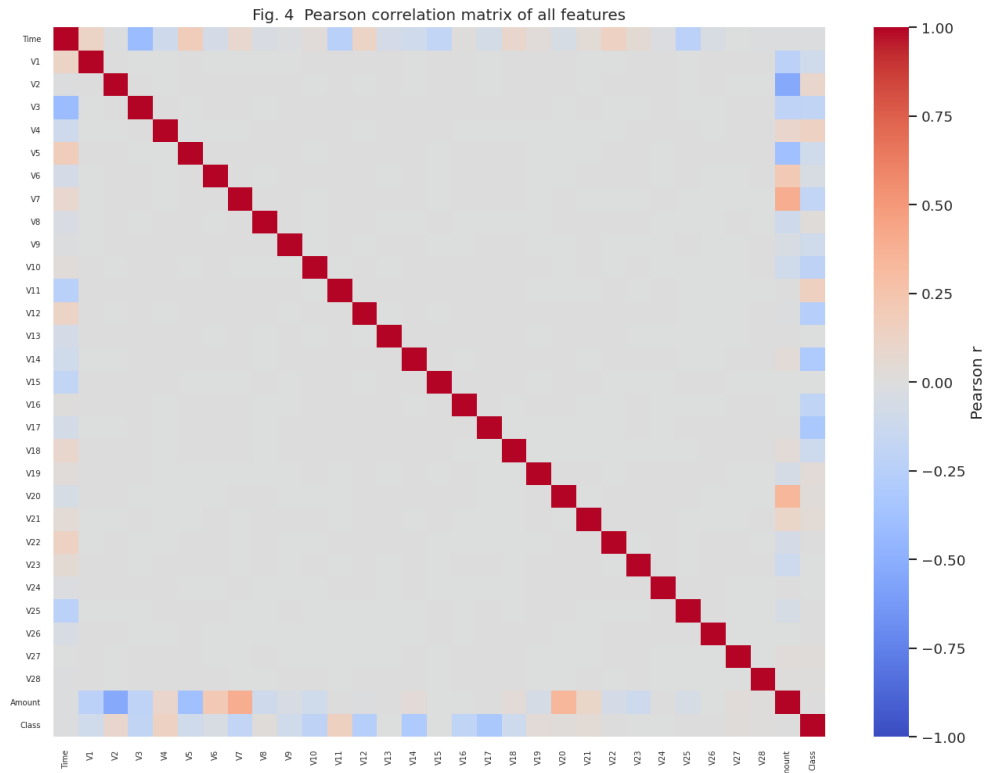


Fig. 4 Pearson correlation matrix of all features. The near-zero off-diagonal structure among V1 through V28 reflects their construction as principal components.

4.5 Baseline and optimisation results

Table 6 reports cross-validated performance for both classifiers with default hyperparameters.

Tab. 6 Baseline performance under 5-fold stratified cross-validation on the training set

Model	Precision	Recall	F1	PR-AUC	ROC-AUC	Time (s)
Logistic Regression (default)	0.8732	0.6294	0.7316	0.7602	0.9778	5.8
XGBoost (default)	0.6760	0.6142	0.6436	0.4481	0.7810	20.1

Source: Own representation.

The untuned XGBoost model underperforms the untuned linear baseline on every metric, which is an unexpected result and is investigated in Section 5.2. Table 7 reports the outcome of the randomised search.

Tab. 7 Randomised search configuration and outcome

Model	Grid size	Sampled	Explored (%)	Best CV F1	Time (s)
Logistic Regression	24	16	66.70	0.7358	347.1
XGBoost	51,840	30	0.06	0.8686	421.6

Source: Own representation.

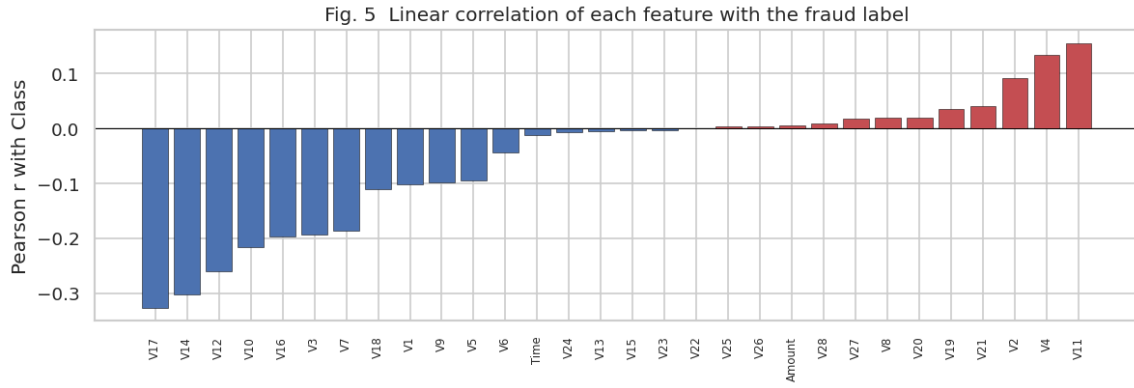


Fig. 5 Linear correlation of each feature with the fraud label.

Figure 6 shows how the search progressed across sampled configurations.

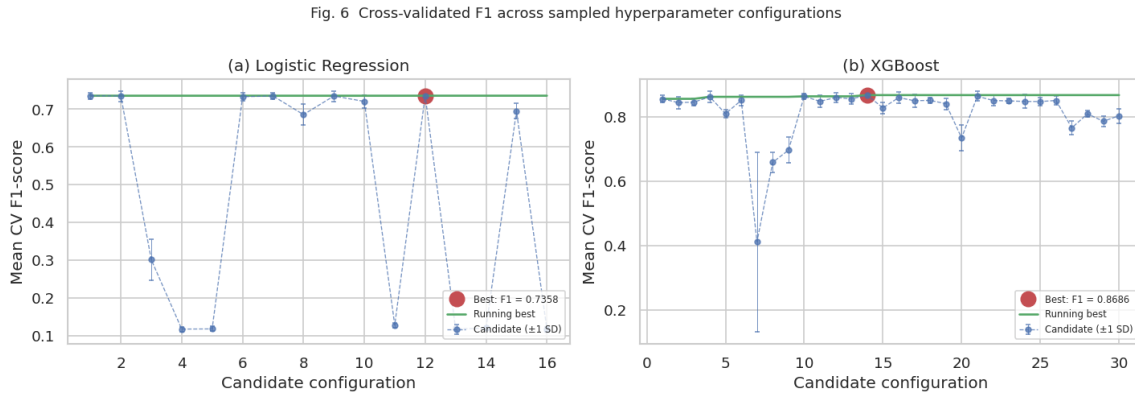


Fig. 6 Cross-validated F1 across sampled hyperparameter configurations. Error bars denote one standard deviation across the five folds; the running-best curve shows the best score attained up to each candidate.

Source: Own representation.

The two panels differ instructively. Logistic Regression candidates cluster within a narrow band, indicating that hyperparameter choice has limited influence on a model whose functional form is fixed. XGBoost candidates span a far wider range, and the running-best curve improves substantially over the sampled sequence, confirming that the configuration of a boosted ensemble materially determines its performance. Despite exploring only 0.06% of its grid, the XGBoost search attained a cross-validated F1 of 0.8686 against 0.7358 for a Logistic Regression search that covered 66.7% of a far smaller space, which is precisely the efficiency property Bergstra and Bengio (2012) describe. Table 8 records the selected configurations.

Source: Own representation. Selection by maximisation of cross-validated F1 on the training partition.

Both searches rejected the imbalance correction, selecting `class_weight = None` and `scale_pos_weight = 1` over the available alternatives. Section 5.3 examines this result.

4.6 Final test-set performance

Table 9 reports the single evaluation on the held-out partition.

Tab. 8 Optimal hyperparameter configurations identified by randomised search

Model	Parameter	Selected value
Logistic Regression	solver	liblinear
	penalty	L1
	C	1
	class_weight	None
XGBoost	n_estimators	500
	max_depth	9
	learning_rate	0.05
	subsample	1.0
	colsample_bytree	0.6
	min_child_weight	5
	gamma	0
	scale_pos_weight	1

Tab. 9 Final classification performance on the held-out test set (n = 56,962; 98 frauds)

Model	Prec.	Recall	F1	PR-AUC	ROC-AUC	TP	FP	FN	TN
LR (default)	0.8267	0.6327	0.7168	0.7414	0.9605	62	13	36	56,851
LR (optimised)	0.8289	0.6429	0.7241	0.7440	0.9576	63	13	35	56,851
XGB (default)	0.8667	0.7959	0.8298	0.7973	0.9390	78	12	20	56,852
XGB (optimised)	0.9412	0.8163	0.8743	0.8751	0.9780	80	5	18	56,859

Source: Own representation. Best value per column in bold.

The optimised XGBoost model leads on every metric of interest. Figure 7 presents the comparison graphically.

**Fig. 7** Test-set performance by model and optimisation state.

Source: Own representation.

The 2x2 design separates two effects. Moving from the linear model to the boosted ensemble raises F1 from 0.7168 to 0.8298 at default settings, a gain of 0.1130 attributable to model family alone. Optimisation then raises XGBoost from 0.8298 to 0.8743, a further 0.0445 attributable to tuning. The corresponding gain for Logistic Regression is 0.0073. Both effects are positive, but they are of different

magnitudes and different causes, and the design permits them to be distinguished.

The confusion matrices in Figure 8 translate these figures into operational terms.

Fig. 8 Confusion matrices on the held-out test set

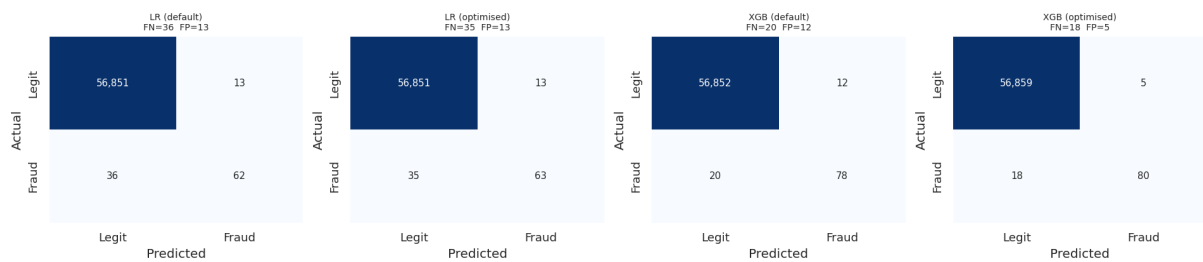


Fig. 8 Confusion matrices on the held-out test set. FN denotes frauds that were not detected; FP denotes legitimate transactions that were wrongly flagged.

Source: Own representation.

The practical difference is substantial. The default Logistic Regression model detects 62 of 98 frauds and misses 36. The optimised XGBoost model detects 80 and misses 18 - half as many failures - while simultaneously reducing false alarms from 13 to 5. It is uncommon for Precision and Recall to improve together, since they usually trade against one another; that both improve indicates a genuinely better ranking of transactions by fraud probability rather than a threshold shift.

4.7 Threshold-independent evaluation

Figure 9 presents the ROC and precision-recall curves, and demonstrates empirically the metric argument advanced in Section 2.3.

Fig. 9 Threshold-independent performance on the test set

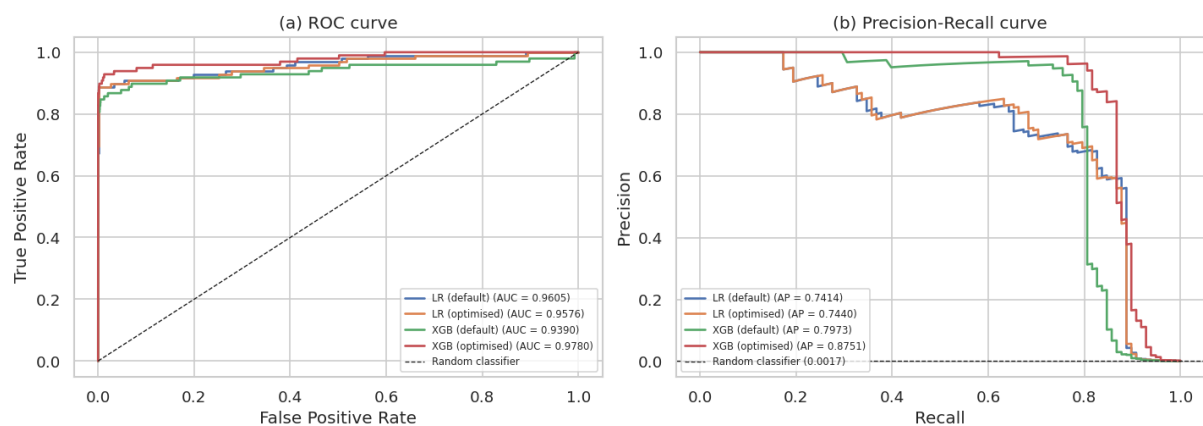


Fig. 9 Threshold-independent performance on the test set. Panel (a) shows ROC curves; panel (b) shows precision-recall curves.

Source: Own representation.

The contrast between the panels is the point. In panel (a) all four curves are compressed into the upper-left corner and are visually almost indistinguishable, with ROC-AUC values spanning only 0.9390 to 0.9780. Read from this panel alone, the default XGBoost model (0.9390) appears *worse* than the default

Logistic Regression model (0.9605), which Table 9 shows to be false by every other measure. In panel (b) the same four models separate clearly, with PR-AUC spanning 0.7414 to 0.8751 and ordering the models consistently with their F1 scores.

The explanation is arithmetical. The false positive rate is $FP / (FP + TN)$, and with 56,864 true negatives available, thirteen false positives yield a rate of 0.00023 while five yield 0.00009. This eightfold difference in absolute false alarms is invisible on a ROC axis. Precision is $TP / (TP + FP)$, and the same difference moves Precision from 0.8267 to 0.9412. This is the effect Saito and Rehmsmeier (2015) describe, reproduced on the present data, and it justifies the decision to treat PR-AUC as the primary threshold-independent metric.

4.8 Threshold sensitivity

The 0.50 decision threshold is a convention inherited from the balanced case and carries no special status here. Figure 10 and Table 10 examine its effect on the best model.

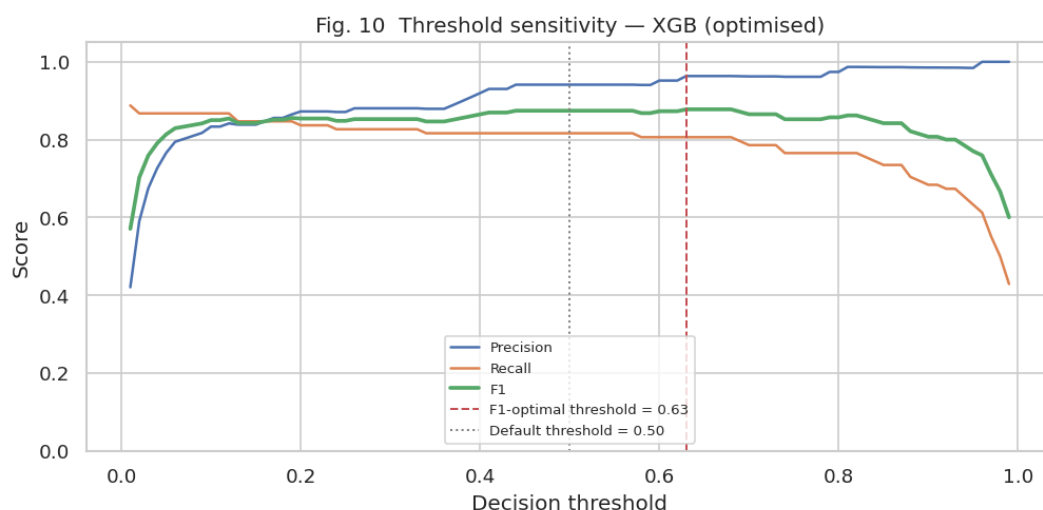


Fig. 10 Threshold sensitivity for the optimised XGBoost model.

Source: Own representation.

Tab. 10 Operational trade-off at selected decision thresholds (optimised XGBoost)

Threshold	Precision	Recall	F1	Caught (TP)	Missed (FN)	False alarms (FP)
0.10	0.8333	0.8673	0.8500	85	13	17
0.25	0.8710	0.8265	0.8482	81	17	12
0.50	0.9412	0.8163	0.8743	80	18	5
0.63	0.9634	0.8061	0.8778	79	19	3
0.75	0.9615	0.7653	0.8523	75	23	3
0.90	0.9853	0.6837	0.8072	67	31	1

Source: Own representation. F1-optimal threshold in bold.

The F1-optimal threshold is 0.63, improving F1 from 0.8743 to 0.8778, a gain of 0.0035. The gain is small, and the more important observation is what the table reveals about the trade-off. Moving from 0.63

to 0.10 catches six additional frauds (79 to 85) at the cost of fourteen additional false alarms (3 to 17). Whether that exchange is worthwhile is not a question F1 can answer, because F1 weights Precision and Recall equally and there is no reason to suppose the underlying costs are equal. A missed fraud incurs the transaction value plus investigation and chargeback costs; a false alarm incurs a blocked customer, a support contact, and reputational damage that is real but harder to price. The threshold is therefore a business decision informed by the model rather than a property of it, and Table 10 is presented as the input to that decision rather than as a recommendation.

4.9 Feature importance

Figure 11 and Table 11 report the features the optimised ensemble relies upon.

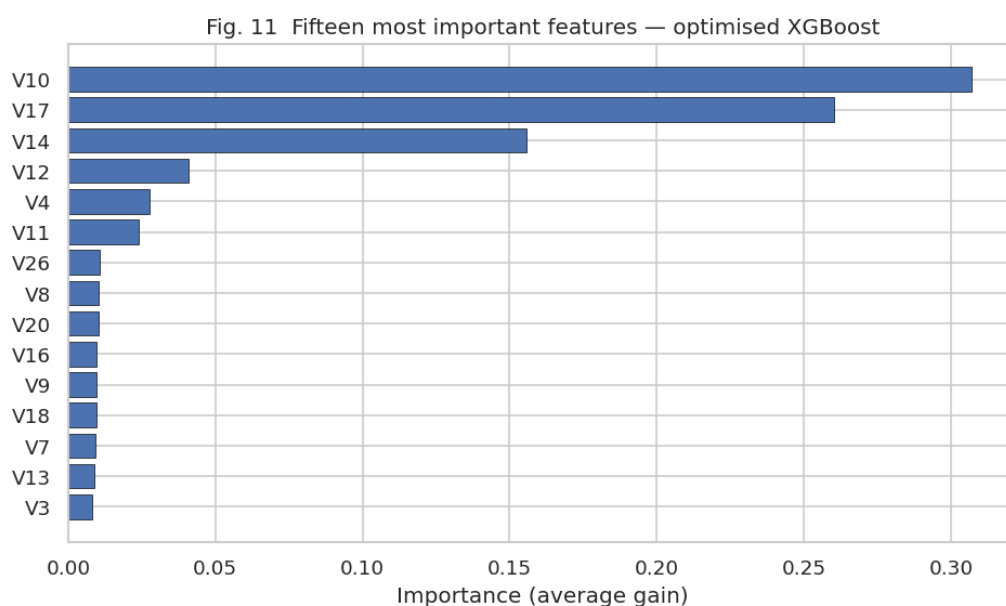


Fig. 11 Fifteen most important features in the optimised XGBoost model, measured by average gain.

Source: Own representation.

Tab. 11 Ten most important features in the optimised XGBoost model

Rank	Feature	Importance (gain)
1	V10	0.3070
2	V17	0.2602
3	V14	0.1556
4	V12	0.0408
5	V4	0.0278
6	V11	0.0238
7	V26	0.0106
8	V8	0.0103
9	V20	0.0103
10	V16	0.0097

Source: Own representation.

The distribution is highly concentrated: V10, V17, and V14 together account for 72.3% of total gain, and the remaining twenty-seven features share the balance. Eleven of the top fifteen also appear among the fifteen features most linearly correlated with the target, indicating substantial but incomplete overlap between what a linear model can exploit and what the ensemble actually uses. The four that appear only in the gain ranking - including V26, V8, and V20, all with near-zero linear correlation - are features whose relationship to the target is conditional on other features and therefore invisible to a correlation coefficient. This is the mechanism underlying the 0.1130 F1 advantage of XGBoost at default settings.

Because V1 through V28 are anonymised principal components, no domain interpretation of these features is available. This is a limitation of the dataset rather than of the analysis, and it is noted rather than speculated around.

5 Discussion

5.1 Model capacity and the value of optimisation

Table 12 summarises the effect of optimisation on both classifiers.

Tab. 12 Effect of hyperparameter optimisation on test-set performance

Model	F1 (def.)	F1 (opt.)	PR-AUC (def.)	PR-AUC (opt.)	Gain	Gain (%)
Logistic Regression	0.7168	0.7241	0.7414	0.7440	0.0073	1.02
XGBoost	0.8298	0.8743	0.7973	0.8751	0.0445	5.36

Source: Own representation.

Optimisation delivered a fivefold greater relative gain for XGBoost than for Logistic Regression. This asymmetry is not incidental and admits a straightforward explanation. Logistic Regression's hyperparameters govern regularisation and the choice of penalty; they cannot alter the fact that the model fits a single hyperplane. When the decision boundary that separates the classes is not a hyperplane, no setting of C will make it one, and the achievable range of performance is correspondingly narrow. The tight clustering of candidates in Figure 6(a) is the visual signature of this constraint.

XGBoost's hyperparameters, by contrast, determine the functional form the ensemble adopts: depth governs the order of interactions representable, the number of rounds and the learning rate jointly determine how much structure is fitted, and the sampling parameters control the bias-variance trade-off. The model is capable of a wide range of behaviours, and the search selects among them.

The practical implication is that optimisation budget should be allocated according to model capacity. Spending compute tuning a linear model on a non-linear problem yields little; the same budget spent on a flexible learner yields substantially more. This is consistent with the emphasis Hajek et al. (2022) and Velarde et al. (2024) place on boosted trees for imbalanced financial classification, though the present study reaches it through a controlled comparison rather than by assumption.

5.2 An underfitting anomaly in the default configuration

Table 6 reports a cross-validated F1 of 0.6436 for the default XGBoost model, while Table 9 reports a test F1 of 0.8298 for the same configuration. The test estimate exceeds the cross-validated estimate by 0.1862. This is the reverse of the expected relationship, and it warranted investigation rather than reporting at face value. The corresponding figures for Logistic Regression are 0.7316 and 0.7168, a gap of -0.0148 in the conventional direction.

The finding was confirmed to be exactly reproducible across independent executions on a freshly initialised environment, agreeing to four decimal places. It is therefore a deterministic property of the configuration rather than a stochastic artefact. Diagnostic checks confirmed that the cross-validation folds were correctly stratified, containing 315 or 316 training frauds and 78 or 79 validation frauds per fold, that feature and label indices were aligned, and that the fraud rate was preserved at 0.1734% against a training-set rate of 0.1729%. The data and the partitioning were sound.

The explanation lies in the interaction between the default configuration and the size of the positive class. XGBoost's defaults specify 100 boosting rounds at a learning rate of 0.3. Each cross-validation fold trains on 182,276 rows containing approximately 315 frauds - 20% fewer positive examples than the 394 available when fitting the full training partition. For a model whose capacity is already constrained by a limited number of rounds, that reduction is consequential, because the number of positives is what bounds the structure the ensemble can learn about the minority class. The default model is underfit, and it is more underfit on a fold than on the full partition. Logistic Regression, which fits four times faster and has far fewer effective parameters, is unaffected by the same reduction.

The search corroborates this reading. It selected 500 rounds at a learning rate of 0.05 - five times the rounds at one sixth the shrinkage - which is precisely the adjustment an underfit model requires. The optimised configuration's cross-validated F1 of 0.8686 and test F1 of 0.8743 differ by only 0.0057, in the expected direction, confirming that once the model has adequate capacity the two estimates converge.

This anomaly is reported rather than suppressed because it constitutes a substantive finding: it demonstrates that library defaults are not neutral, that their adequacy is contingent on the quantity of minority-class data available, and that a cross-validated estimate of an underfit model systematically understates what that model achieves when fitted on more data. Had only cross-validation been reported, XGBoost would have appeared inferior to the linear baseline, and a reader would have drawn the opposite conclusion from the correct one. This is the concrete value of the dual-protocol design described in Section 3.3.

5.3 The rejection of cost-sensitive weighting

Both searches declined the imbalance correction. The XGBoost grid offered `scale_pos_weight` values of 1, 24.03, and 577.29, and the search selected 1. The Logistic Regression grid offered `class_weight` of None and balanced, and the search selected None. Neither result was imposed; both emerged from maximisation of cross-validated F1, and both are reproducible.

The result is initially counterintuitive but follows from the choice of criterion. Setting `scale_pos_weight` to 577.29 makes a single fraud as costly to misclassify as 577 legitimate

transactions. The optimiser responds by pushing the decision boundary aggressively toward the majority class, which raises Recall and depresses Precision. F1, weighting the two equally, penalises the resulting imbalance. The correction does what it is designed to do; F1 simply does not reward it.

Two considerations qualify the finding. First, it is conditional on the metric. Had the search been scored on Recall, or on a cost-weighted criterion assigning a missed fraud several times the cost of a false alarm, a non-unit weight would very plausibly have been selected. The result therefore says less about imbalance correction in general than about what F1 rewards under this data distribution. Second, it is conditional on the separability established in Section 4.3. The evidence of Figure 3 is that the classes occupy distinguishable regions of the principal component space; a sufficiently flexible model can locate the boundary without having the loss function reweighted to draw its attention there. The correction addresses a problem the optimised ensemble does not have.

The wider implication is that cost-sensitive weighting is not automatically beneficial merely because a dataset is imbalanced. Whether it helps depends jointly on the separability of the classes, the capacity of the model, and the criterion by which success is judged. Placing the parameter under search rather than fixing it at the theoretically motivated value is what allowed this to be observed rather than assumed.

5.4 Comparison against the literature

Velarde et al. (2024) find tree boosting methods to be robust on imbalanced risk assessment tasks, and Hajek et al. (2022) report strong performance for an XGBoost-based framework on payment fraud. The present results are consistent with both: the optimised XGBoost model outperforms the linear baseline by 0.1502 in F1 and 0.1311 in PR-AUC.

One qualification is warranted regarding the robustness claim. The results here indicate that XGBoost is robust *once configured*. In its default configuration it was outperformed by Logistic Regression under cross-validation, for the reasons set out in Section 5.2. The robustness that the literature attributes to boosted trees should therefore be understood as a property of the model class given adequate tuning, not of the library defaults. For practitioners this distinction has consequences: adopting XGBoost without tuning may yield worse results than a well-specified linear model, particularly when the minority class is small.

5.5 Limitations

Five limitations bound the interpretation of these results.

The dataset records two days of transactions from September 2013. Fraud patterns evolve as adversaries adapt to deployed defences, and a model trained on this distribution should not be assumed to transfer to contemporary transaction streams. Velarde et al. (2024) address robustness over time explicitly, and the single fixed partition used here cannot speak to it. Temporal validation, in which the model trains on earlier transactions and is evaluated on later ones, would provide a more realistic estimate of deployed performance.

The features are anonymised principal components. This precludes domain interpretation of the impor-

tance ranking in Table 11, and it precludes feature engineering, which in operational fraud detection is often the largest source of gain - velocity features, merchant category aggregates, and deviation from a cardholder's historical behaviour cannot be constructed from V1 through V28.

The test partition contains 98 frauds. A single additional detection moves Recall by 0.0102, so the differences reported between closely matched configurations should not be over-read. Reporting cross-validated standard deviations alongside the point estimates, or bootstrapping confidence intervals on the test metrics, would quantify this uncertainty; the present study reports the fold-level spread in Figure 6 but does not extend it to the test set.

The XGBoost search sampled 30 of 51,840 possible configurations, or 0.06% of the grid. Bergstra and Bengio (2012) support the adequacy of sparse random sampling, and the plateau visible in the running-best curve of Figure 6(b) is consistent with the budget being sufficient. A larger budget might nonetheless locate a better configuration, and no claim of global optimality is made.

Finally, imbalance was addressed only through cost-sensitive weighting. Resampling methods, including SMOTE (Chawla et al., 2002) and the hybrid strategies catalogued by Lemaître et al. (2017), were not evaluated. Given that the search rejected even the cost-sensitive correction, resampling appears unlikely to help under an F1 criterion on this data, but that expectation is untested here.

6 Conclusion

This study evaluated two classifiers of contrasting inductive bias on the Credit Card Fraud Detection dataset, a binary classification problem exhibiting an imbalance ratio of 1:578. Both were embedded in leakage-controlled pipelines, evaluated under 5-fold stratified cross-validation and on a held-out partition, and optimised by randomised parameter search.

Five conclusions follow from the results.

The optimised XGBoost model achieved the strongest performance, with Precision = 0.9412, Recall = 0.8163, F1 = 0.8743, and PR-AUC = 0.8751 on the held-out test set. It detected 80 of 98 frauds while raising 5 false alarms among 56,864 legitimate transactions, improving on the linear baseline in both Precision and Recall simultaneously - an indication of better probability ranking rather than a threshold shift.

The value of hyperparameter optimisation scales with model capacity. XGBoost gained 5.36% in F1 against 1.02% for Logistic Regression, because the former's hyperparameters determine its functional form while the latter's merely regularise a fixed one. Optimisation budget is best allocated to flexible learners.

The default XGBoost configuration proved markedly underfit when trained on cross-validation folds, scoring 0.6436 in cross-validation against 0.8298 on the test set. The effect was exactly reproducible and traced to the interaction between 100 boosting rounds at a learning rate of 0.3 and the reduced number of positive examples available per fold. Library defaults are not neutral, and their adequacy depends on the quantity of minority-class data.

Cost-sensitive weighting was rejected by both searches under an F1 criterion. The correction trades

Precision for Recall, and F1 weights them equally; combined with the demonstrated separability of the classes in the principal component space, the optimiser found no benefit in it. Imbalance correction is not automatically beneficial merely because data are imbalanced, and its value is contingent on the evaluation criterion.

Metric choice materially affects conclusions. ROC-AUC ranked the default XGBoost model below the default Logistic Regression model, which every other measure contradicts. PR-AUC ordered the models consistently with F1. Under extreme imbalance, ROC-AUC should be reported only with the caution its denominator warrants.

Three directions would extend this work. Temporal validation would test whether the observed performance survives distributional drift, addressing the concern Velarde et al. (2024) raise. A cost-weighted evaluation criterion, replacing F1 with an objective reflecting the actual asymmetry between a missed fraud and a blocked customer, would very likely reverse the finding on cost-sensitive weighting and would better reflect the deployment context. And an explicit comparison against resampling strategies would establish whether the rejection of cost-sensitive weighting generalises to data-level imbalance corrections.

Reproducibility. All experiments fixed `random_state = 42` for the partition, the cross-validation folds, both estimators, and the search sampler. All preprocessing was encapsulated within pipeline objects, precluding fold-level leakage. The environment comprised scikit-learn 1.6.1 and xgboost 3.2.0 on Python 3.12. The complete executable notebook, including all figures and tables reported here, is available at:

<https://github.com/BijayKhanalDS/Credit-card-fraud-detection>

References

- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281-305.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Hajek, P., Abedin, M. Z., & Sivarajah, U. (2022). Fraud detection in mobile payment systems using an XGBoost-based framework. *Information Systems Frontiers*, 24, 1-19. <https://doi.org/10.1007/s10796-022-10346-6>
- Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1-5. <http://jmlr.org/papers/v18/16-365.html>
- Machine Learning Group - ULB. (n.d.). *Credit card fraud detection*. Kaggle. <https://www.kaggle.com/mlg-ulb/creditcardfraud>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. https://scikit-learn.org/stable/model_selection.html
- Pozzolo, A. D., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence* (pp. 159-166). IEEE. <https://doi.org/10.1109/SSCI.2015.33>
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Velarde, G., Weichert, M., Deshmunkh, A., Deshmane, S., Sudhir, A., Sharma, K., & Joshi, V. (2024). Tree boosting methods for balanced and imbalanced classification and their robustness over time in risk assessment. *Intelligent Systems with Applications*, 21, Article 200354. <https://doi.org/10.1016/j.iswa.2024.200354>

Appendix A: Python Notebook Code and Results

The complete executable notebook, containing all code, outputs, figures, and tables reported in this study, is available in the following repository:

<https://github.com/BijayKhanalDS/Credit-card-fraud-detection>

The notebook was executed on the Kaggle platform against the Credit Card Fraud Detection dataset published by the Machine Learning Group at the Université Libre de Bruxelles. A printed copy of the executed notebook, including all cell outputs, is appended following this page.

A.1 Software environment

Component	Version
Python	3.12
scikit-learn	1.6.1
xgboost	3.2.0
pandas	2.x
numpy	2.x
matplotlib	3.x
seaborn	0.13.x
random_state	42

A.2 Dataset summary

Property	Value
Shape	284,807 rows by 31 columns
Legitimate transactions	284,315 (99.827%)
Fraudulent transactions	492 (0.173%)
Missing values	0
Duplicate rows	1,081 (retained)
Imbalance ratio	1:578
Observation window	48.0 hours

A.3 Selected hyperparameters

Model	Parameter	Value
Logistic Regression	solver	liblinear
	penalty	L1
	C	1
	class_weight	None
	Best CV F1	0.7358
XGBoost	n_estimators	500
	max_depth	9
	learning_rate	0.05
	subsample	1.0
	colsample_bytree	0.6
	min_child_weight	5
	gamma	0
	scale_pos_weight	1
	Best CV F1	0.8686

A.4 Final test-set results

Model	Precision	Recall	F1	PR-AUC
LR (default)	0.8267	0.6327	0.7168	0.7414
LR (optimised)	0.8289	0.6429	0.7241	0.7440
XGB (default)	0.8667	0.7959	0.8298	0.7973
XGB (optimised)	0.9412	0.8163	0.8743	0.8751